

IPC – Casos de Aplicación



```
$ ./anotador &  
$ ./pantalla  
3 - 1
```

Contenido

1

Memoria compartida

Un tablero de marcador en tiempo real.

2

Semáforos de proceso

Un estacionamiento de cupo limitado,

3

Memoria mapeada

El récord de un arcade.

4

Pipes y FIFOs

Entrega de pedidos de cocina

5

Sockets

Pedidos en una heladería: mostrador o delivery

MEMORIA COMPARTIDA

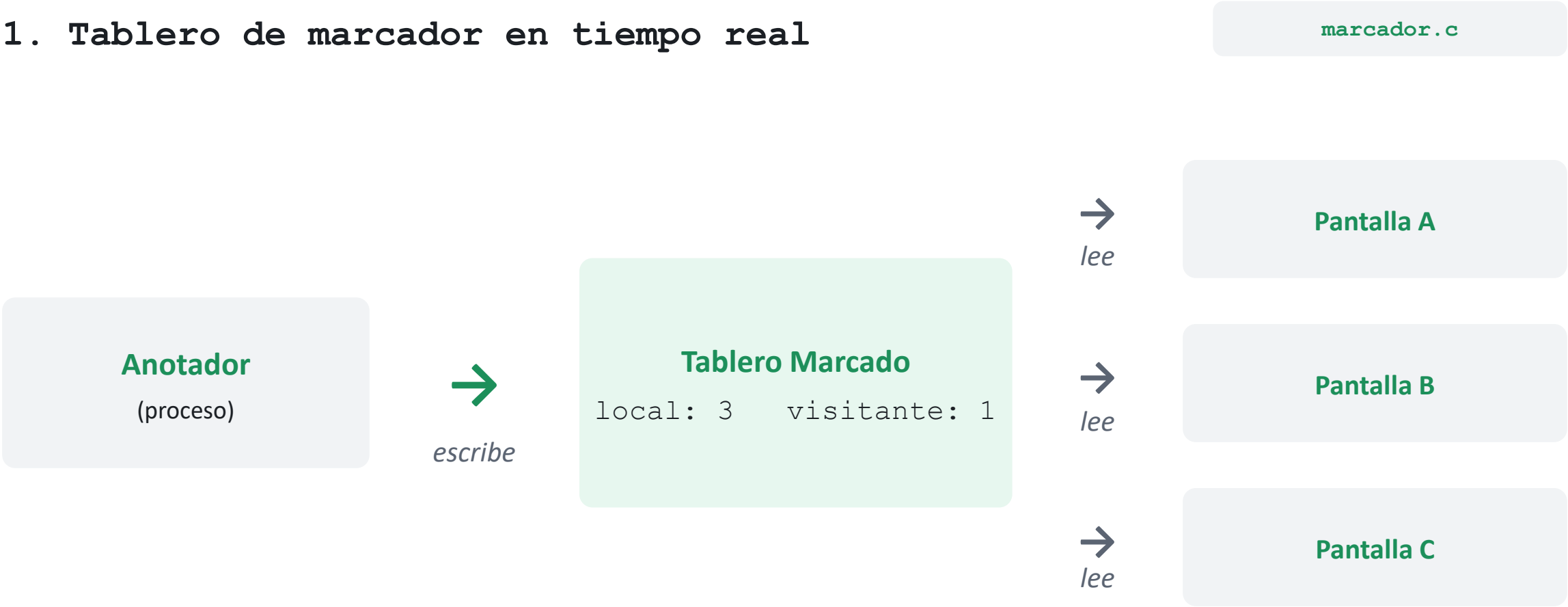
marcador.c

1. Tablero de marcador en tiempo real

1. Un partido en vivo tiene un anotador que lleva el resultado, y varias pantallas (Cartelera A, Cartelera B, Cartelera C) que deben mostrarlo actualizado todo el tiempo.
2. Copiar el marcador en cada pantalla por separado, cada vez que el mismo cambia, sería lento y se podría notar la latencia de actualización.
3. **Solución:** Si todos los procesos comparten el mismo bloque de memoria, el anotador escribe allí, y cada pantalla lee directamente, sin demora.
4. *Escenario apropiado cuando varios procesos en la misma máquina necesitan ver datos que cambian seguido "al instante".*

MEMORIA COMPARTIDA

1. Tablero de marcador en tiempo real



MEMORIA COMPARTIDA

1. Tablero de marcador en tiempo real

marcador.c

```
  
// anotador.c - escribe el marcador  
#include <sys/shm.h>  
struct Marcador { int local, visitante; };  
int id = shmget(0x1234, sizeof(struct Marcador), IPC_CREAT | 0666);  
struct Marcador* m = (struct Marcador*) shmat(id, NULL, 0);  
m->local = 3;  
m->visitante = 1;           // cualquier pantalla lo ve al instante  
shmdt(m);  
  
// pantalla.c - lee el marcador (misma clave 0x1234)  
int id = shmget(0x1234, sizeof(struct Marcador), 0666);  
struct Marcador* m = (struct Marcador*) shmat(id, NULL, 0);  
printf("%d - %d\n", m->local, m->visitante);  
shmdt(m);
```

SEMÁFOROS DE PROCESO

2. Estacionamiento de capacidad limitada

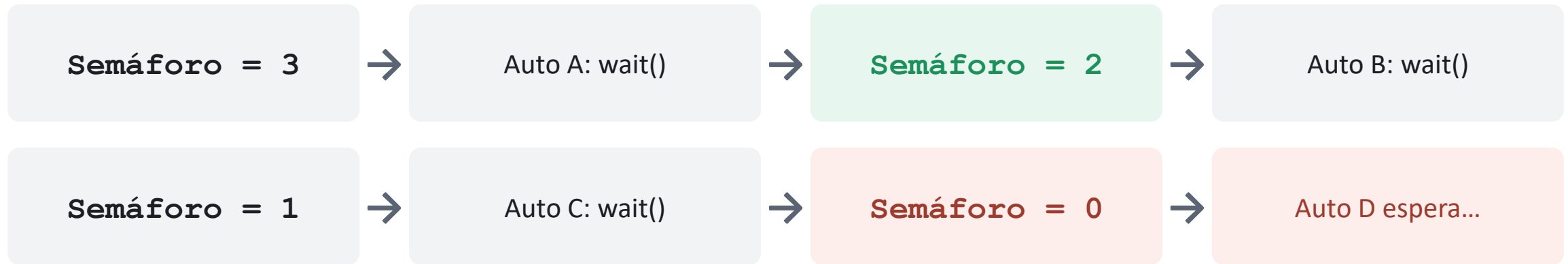
`estacionamiento.c`

1. Un estacionamiento tiene 3 lugares. Varios autos (procesos) llegan al mismo tiempo y necesitan saber si hay lugar, sin que dos autos “asuman” que el mismo lugar es suyo.
2. Un semáforo se inicializa en 3 tickets. Luego, cada auto que ingresa hace `wait()` y resta 1; cada auto que sale hace `signal()` y suma 1.
3. Cuando el semáforo llega a 0, el próximo auto que intente ingresar queda esperando automáticamente, ya que el kernel lo bloquea. No se precisa preguntar en un bucle.
4. Escenario que ocurre cuando varios procesos comparten un recurso limitado pero no de a uno, sino de a un cupo de N , y se debe evitar que lo excedan.

SEMÁFOROS DE PROCESO

2. Estacionamiento de capacidad limitada

estacionamiento.c



→ El SO despierta al Auto D cuando corresponde

En cuanto algún Auto sale y realiza un signal(), entonces el semáforo sube a 1 y el SO selecciona a un proceso en espera (por ejemplo el Auto D) para reactivarlo y que continúe.

SEMÁFOROS DE PROCESO

2. Estacionamiento de capacidad limitada

estacionamiento.c

```
// estacionamiento.c - semáforo contador con 3 lugares
#include <sys/sem.h>
int sem_id = semget(IPC_PRIVATE, 1, IPC_CREAT | 0666);
semctl(sem_id, 0, SETVAL, 3);      // 3 lugares libres al inicio

void entrar(void) {
    struct sembuf op = { 0, -1, SEM_UNDO };
    semop(sem_id, &op, 1); // equivale a un wait().
                          // Resta 1 al semáforo, suspende el proceso si hace falta
}

void salir(void) {
    struct sembuf op = { 0, +1, SEM_UNDO };
    semop(sem_id, &op, 1); // Equivale a un signal(). Suma 1 al semáforo, nunca bloquea al proceso
}
```

MEMORIA MAPEADA

3. Record de Arcade

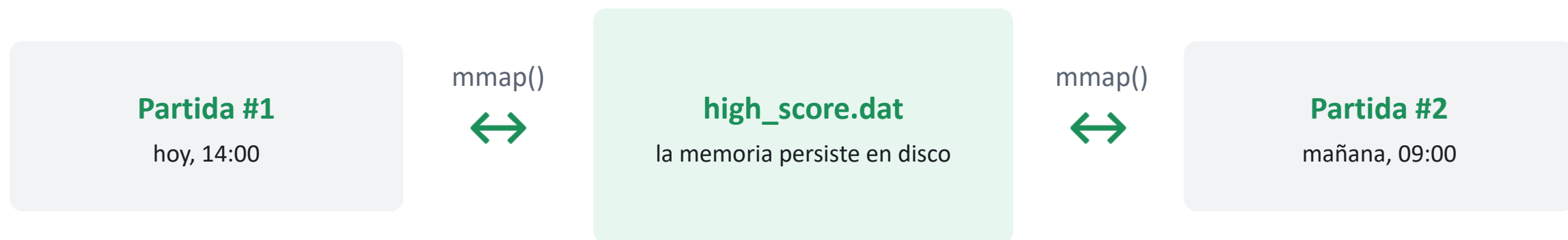
arcade.c

1. Un juego de arcade guarda el puntaje más alto (record). Cada partida es un proceso nuevo y todas deben ver (y poder mejorar) el récord, incluso tras reiniciar la máquina.
2. A diferencia del Marcador del Caso 1, el dato debe persistir aun aunque el proceso que lo escribió haya finalizado y es por ellos que la memoria compartida "pura" no es suficiente.
3. `mmap()` asocia un archivo (`high_score.dat`) a la memoria. Al leer o escribir esa memoria, se lee o escribe el archivo automáticamente.
4. Resulta apropiado cuando el dato compartido debe persistir entre proceso y sus ejecuciones, o cuando conviene tratar un archivo como una simple variable.

MEMORIA MAPEADA

3. Record de Arcade

`arcade.c`



Dos partidas pueden o no correr al mismo tiempo. El archivo en disco preserva la memoria.

MEMORIA MAPEADA

3. Record de Arcade

arcade.c

```
// arcade.c - puntaje más alto, compartido vía archivo mapeado
#include <fcntl.h>
#include <sys/mman.h>

int fd = open("high_score.dat", O_RDWR | O_CREAT, 0644);
ftruncate(fd, sizeof(int));           // asegurar el tamaño
int* record = mmap(NULL, sizeof(int), PROT_READ | PROT_WRITE,
                    MAP_SHARED, fd, 0);

close(fd);                           // el mapeo sigue vivo

int puntaje = jugar();                // el jugador termina la partida
if (puntaje > *record)
    *record = puntaje;                // el archivo cambia solo
munmap(record, sizeof(int));
```

PIPES Y FIFOS

4. Entrega de pedidos

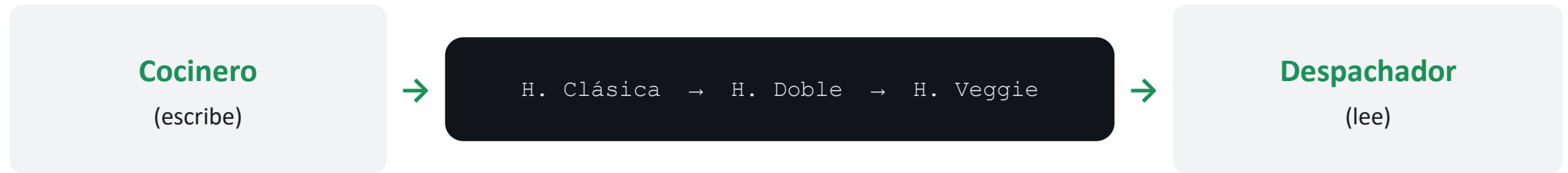
`pedidos.c`

1. En una cocina, un cocinero prepara los pedidos y un despachador los entrega. El despachador solo puede entregar los pedidos que el cocinero ya ha preparado, en el mismo orden en que los realizó.
2. Un pipe establece un canal de dos extremos. Lo que el cocinero entrega de un lado, el despachador lo recibe del otro, en orden.
3. Cocinero y despachador deben ser procesos emparentados. Un pipe se inicializa antes del `fork()`, de forma a que ambos hereden sus extremos.
4. Es apropiado para dos procesos relacionados que se pasan datos en un solo sentido, en orden, sin que requerimiento de coordinar la velocidad de cada uno.

PIPES Y FIFOs

4. Entrega de pedidos

`pedidos.c`



Los pedidos salen en el mismo orden en que se preparan. El pipe no los mezcla ni los adelanta.

→ ¿Y si los procesos no están relacionados entre si ?

`mkfifo("/tmp/pedidos")` crea el mismo tipo de canal, pero con un nombre en el filesystem — así dos programas distintos no relacionados (sin `fork()` de por medio) pueden ejecutarse de forma equivalente.

PIPES Y FIFOS

4. Entrega de pedidos

pedidos.c

```
// pedidos.c - cocinero y empacador, comunicados por un pipe
#include <unistd.h>
#include <string.h>

int fds[2];
pipe(fds); // fds[0]=lectura, fds[1]=escritura

if (fork() == 0) {
    close(fds[1]); // el empacador no escribe
    char pedido[32];
    while (read(fds[0], pedido, sizeof(pedido)) > 0)
        printf("Empacando: %s\n", pedido);
} else {
    close(fds[0]); // el cocinero no lee
    char* pedidos[] = { "clasica", "doble", "veggie" };
    for (int i = 0; i < 3; i++)
        write(fds[1], pedidos[i], strlen(pedidos[i]) + 1);
    close(fds[1]); // avisa fin de pedidos
}
```

SOCKETS

5. La heladería

heladeria.c

1. Una heladería recibe pedidos de dos formas: en el mostrador (mismo local) y por teléfono. En ambos casos, alguien pide y el negocio responde.
2. La heladería (servidor) abre un socket, lo asocia a una dirección, escucha y recibe pedidos a medida que llegan. Cada cliente se conecta a la dirección realizar un pedido.
3. La heladería y el cliente no están emparentados ni comparten memoria, ninguno tiene noción del otro hasta que interactúan.
4. Escenario apropiado cuando procesos no relacionados pueden, eventualmente, residir en máquinas distintas. Los eventos de comunicación pueden ser locales o externos, pero por el mismo canal.

SOCKETS

5. La heladería

heladeria.c

Cliente

(cliente)

pedido: "1 cucurucho de chocolate"



respuesta: "\$500 · listo en 5 min"



Heladería

(servidor)

PF_LOCAL

Mostrador — mismo local

PF_INET

Teléfono — externo

SOCKETS

5. La heladería

heladeria.c

```
// heladeria.c - servidor: atiende pedidos
#include <sys/socket.h>
int fd_local = socket(PF_LOCAL, SOCK_STREAM, 0); // mostrador
int fd_inet = socket(PF_INET, SOCK_STREAM, 0); // teléfono
bind(fd_local, ...); listen(fd_local, 5);
bind(fd_inet, ...); listen(fd_inet, 5);

int max_fd = (fd_local > fd_inet ? fd_local : fd_inet) + 1;

while (1) {
    fd_set listos;
    FD_ZERO(&listos);
    FD_SET(fd_local, &listos);
    FD_SET(fd_inet, &listos);
    select(max_fd, &listos, NULL, NULL, NULL);
    if (FD_ISSET(fd_local, &listos)) atender(accept(fd_local, NULL, NULL));
    if (FD_ISSET(fd_inet, &listos)) atender(accept(fd_inet, NULL, NULL));
}
```